Python

Installation, environment and IDE setup

Installing Python

- MacOS & Linux: built-in Python 3 from the OS (via terminal)
  - you get older versions
  - for newer versions: do **NOT** upgrade your OS Python!
    (mini)Conda
    (micro)Mamba
    pixi
- Windows WSL: no Python – Install miniconda/micromamba/pixi
- Windows:
  - Download Python (only from the official website), or
  - Install miniconda/micromamba/pixi (recommended)

Miniconda Installation (MacOS & Linux)

- Open your terminal -> `cd` to a temp directory

```
wget https://repo.continuum.io/miniconda/Miniconda3-latest-Linux-x86_64.sh

DIR_INSTALL=/path/to/my/local/appdir

bash Miniconda3-latest-Linux-x86_64.sh -b -p ${DIR_INSTALL}
rm -f Miniconda3-latest-Linux-x86_64.sh
```

Miniconda Installation (Windows)

- Open file browser -> go to a temp directory
- Right click -> Open in Terminal
- In PowerShell:

```
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-Windows-x86_64.exe -o .\miniconda.exe
.\miniconda.exe
del .\miniconda.exe          # delete once done
```

- Choose an installation folder, e.g.
  `C:\Workdir\myapps\miniconda`
- Choose install options, specially ‘Add Anaconda to my **PATH** …’

Miniconda Installation (Windows)

- Open PowerShell and try: `conda –version`
  if it works: `conda init powershell`
- [Troubleshooting]:
- Elevate privileges to administrator
- `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`
- `Get-ExecutionPolicy –List`
- `notepad $PROFILE`
- `$env:Path += ";C:\Workdir\myapps\miniconda;C:\Workdir\myapps\miniconda\Scripts;C:\Workdir\myapps\miniconda\Library\bin"`

Miniconda Environment Management

- You can reproduce your work by using the same environment
- You can share your environment with the world to reproduce your results
- Create a new environment for a new project
- Create a new environment to update package versions
- Avoid using the ‘base’ environment
- Given your environments informative names, e.g.
  Avoid: work, science, project, thesis …
  Use: py3.13-pytorch-gpu-2.8, …

Miniconda Environment Creation

- Windows: Open PowerShell (as Administrator?)

```
conda create python==3.13 pandas==2.3.3 \
      -p C:\Workdir\myapps\miniconda\envs\py3.13-pd2.3.3
conda activate C:\Workdir\myapps\miniconda\envs\py3.13-
pd2.3.3
```

- MacOS/Linux:

```
$ conda create -n py3.13-pd2.3.3 python==3.13 \
                  pandas==2.3.3
$ conda activate py3.13-pd2.3.3
```

Miniconda Environment Usage

- From now it, using Conda is OS-agnostic

```
> conda activate py3.13-pd2.3.3
> python
>>> import pandas as pd
>>> pd.__file__
'C:\\Workdir\\myapps\\miniconda\\envs\\py3.13-
pandas2.3.3\\Lib\\site-packages\\pandas\\__init__.py'
>>> pd.__version__
‘2.3.3’
```

If you succeed up to here … Congratulations then!

Environment: general workflow (CLI)

1. Activate the Environment:
   `conda activate py3.13-pd2.3.3`
2. Install additional packages:
   `conda install matplotlib`
3. Work on Your Project:
   With the environment activated, you can run your Python scripts and work on your project. All installed packages will be isolated to this environment.
4. Deactivate the Environment when done:
   `conda deactivate`

- Using environments helps keep your projects organized and ensures that dependencies don't interfere with each other.

Environment: extra (CLI)

- Export a current environment to a .yml file:
  `conda env export --name py3.13-pd2.3.3> py3.13-pd2.3.3.yml`
- Create an exact copy of an existing environment:
  `conda create –f py3.13-pd2.3.3.yml`
- List all current existing environments:
  - `conda env list`
- List all the packages and their versions installed in your current conda environment.
  - `conda list`
- To completely delete an enviroment, use the command:
  - `conda remove --name ENV_NAME`

Running Python Code

Some background

- https://docs.anaconda.com/anaconda/getting-started/
- https://realpython.com/run-python-scripts/
- https://plot.ly/python/ipython-vs-python/
- https://yihui.name/en/2018/09/notebook-war/
- https://www.theatlantic.com/science/archive/2018/04/the-scientific-paper-is-obsolete/556676/
- https://fangohr.github.io/blog/installation-of-python-spyder-numpy-sympy-scipy-pytest-matplotlib-via-anaconda.html

Making it work

- Write the code
  - Choose a good editor (or integrated development environment - IDE)
    - featuring color coding, syntax checks, code completion, function call help, AI integration, Git integration, …
  - Code is just a text file
- Convert to machine code
  - Make sure that you have the right interpreter (or compiler) available
- Run the code
  - Run interactively on the Python command line (e.g. IPython, Python shell)
  - Run in a script mode (e.g. `python analysis.py`)
  - Run in IDE or in Jupyter notebooks

Where to start?

- Choose a platform - primary ways to run Python code:
    1. Terminal
        1. Python interpreter
        2. IPython interpreter
        3. Running scripts
    2. IDE
        - Visual Studio Code (Microsoft)
        - Spyder (JetBrains)
    3. Jupyter notebook / Jupyterlab

Hello World

How to run Hello World code?

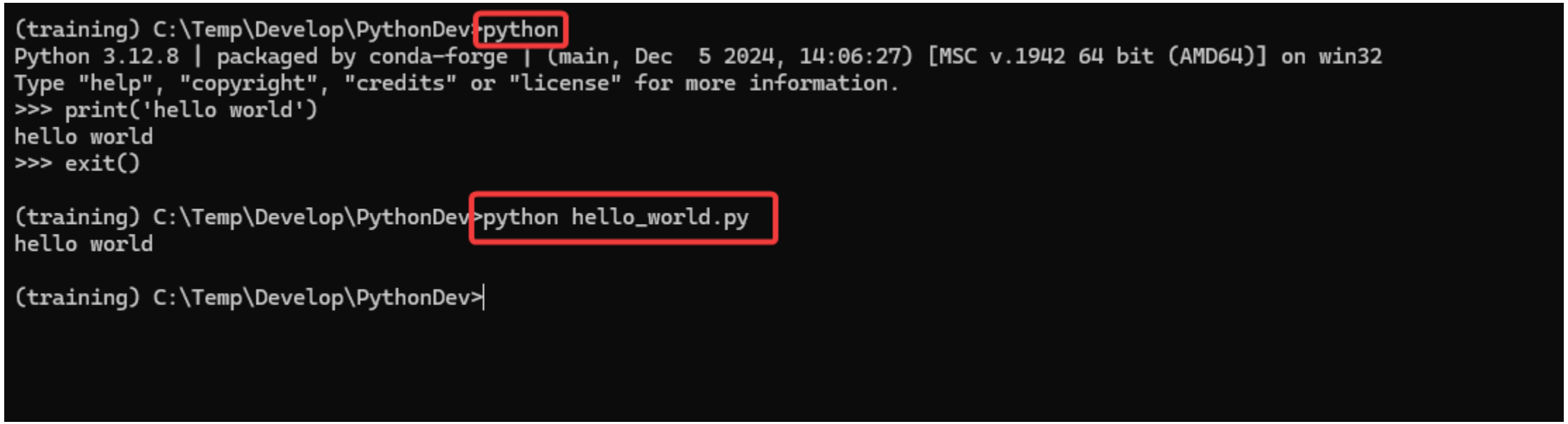
- Interactively: `print('Hello World')` in python interpreter
- `python hello_world.py`
- Run in IDE
  - `%run hello_world.py`
- Run in Jupyter notebook

Python interpreter

https://realpython.com/run-python-scripts/

Python interpreter

- The interpreter is able to run Python code in two different ways:
  - As a piece of code typed into an interactive session
  - As a script or module



Python interpreter

- The most basic way to execute Python code is line by line within the Python interpreter (interactive session).
- The Python interpreter can be started by typing: `python`
  - Terminal on Mac OS X and Unix/Linux systems,
  - Command Prompt application in Windows
  - >>> by default

```
(training) C:\Temp\Develop\PythonDev>python
Python 3.12.8 | packaged by conda-forge | (main, Dec  5 2024, 14:06:27) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('hello world')
hello world
>>> a = 1
>>> b = 2.3
>>> c = a / b
>>> print(c)
0.4347826086956522
>>>
```

IPython interpreter

- Enhanced Interactive shell
- Enhancements to the basic Python interpreter: `ipython`
- https://stackoverflow.com/questions/12370457/what-is-the-difference-between-python-and-ipython

```
(training) C:\Temp\Develop\PythonDev>ipython
Python 3.12.8 | packaged by conda-forge | (main, Dec  5 2024, 14:06:27) [MSC v.1942 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.31.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: print('hello world')
hello world

In [2]: a = 1

In [3]: b = 2.3

In [4]: c = a / b

In [5]: print(c)
0.4347826086956522

In [6]:
```

IPython interpreter

- IPython is an enhanced version of python that makes interactive python more productive.
  - Tab autocompletion (on class names, functions, methods, variables)
  - More explicit and color-highlighted error messages
  - Better history management
  - Basic UNIX shell integration (run simple shell commands such as cp, ls, rm, cp, etc. directly from the IPython command line)
  - Nice integration with many common GUI modules (PyQt, PyGTK, and tkinter)
  - https://www.quora.com/What-is-the-difference-between-IPython-and-Python-Why-would-I-use-IPython-instead-of-just-writing-and-running-scripts

Magic commands

- https://ipython.readthedocs.io/en/stable/interactive/magics.html
- An enhancement in IPython known as magic commands
- Get more information: `%magic`
- Information of a specific magic function is obtained by `%magicfunction?`
- Quick list of all available magic functions: `%lsmagic`

Magic commands

- Designed to solve various common problems in standard data analysis.
- 2 types
  - Line magics
    - They are similar to command line calls. They start with % character. Rest of the line is its argument passed without parentheses or quotes.
  - Cell magics
    - They have %% character prefix. Unlike line magic functions, they can operate on multiple lines below their call.

Python scripts

- Programs: save code to file, and execute it all at once.
  - Script: A plain text file containing Python code that is intended to be directly executed by the user
  - By convention, Python scripts are saved in files with a *.py* extension.

```
1  print('hello world')
2  a = 1
3  b = 2.3
4  c = a / b
5  print(c)
```

```
(training) C:\Temp\Develop\PythonDev>python python_test.py
hello world
0.43478260869565222

(training) C:\Temp\Develop\PythonDev>
```

Run Python script

Linux

- Write script in editor
  Add shebang on line 1:
  `#!/usr/bin/env python3`
- Run script using Python interpreter
  `python hello_world.py`
- Make script executable
- `chmod u+x hello_world.py`
- Run script directly
  `./hello_world.py`

Windows

- Write script in editor
- Run script using Python interpreter
  `python hello_world.py`
- Run script directly
  `hello_world.py`

Python scripts

- Linux

```
#!/usr/bin/env python
```

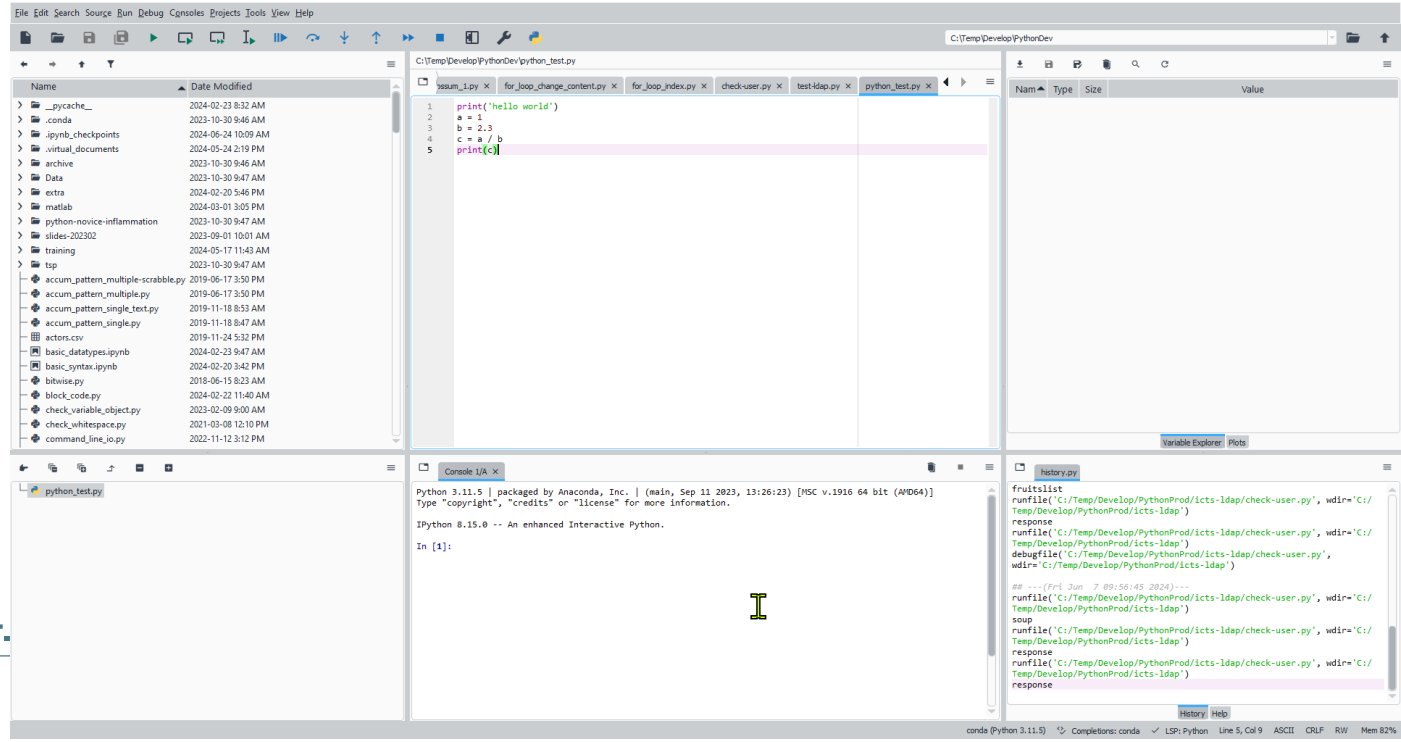
  - determines the script's ability to be executed like a standalone executable without typing `python` in the terminal
  - double clicking it in a file manager (when configured properly).

Spyder

Another IDE

IDE: Spyder

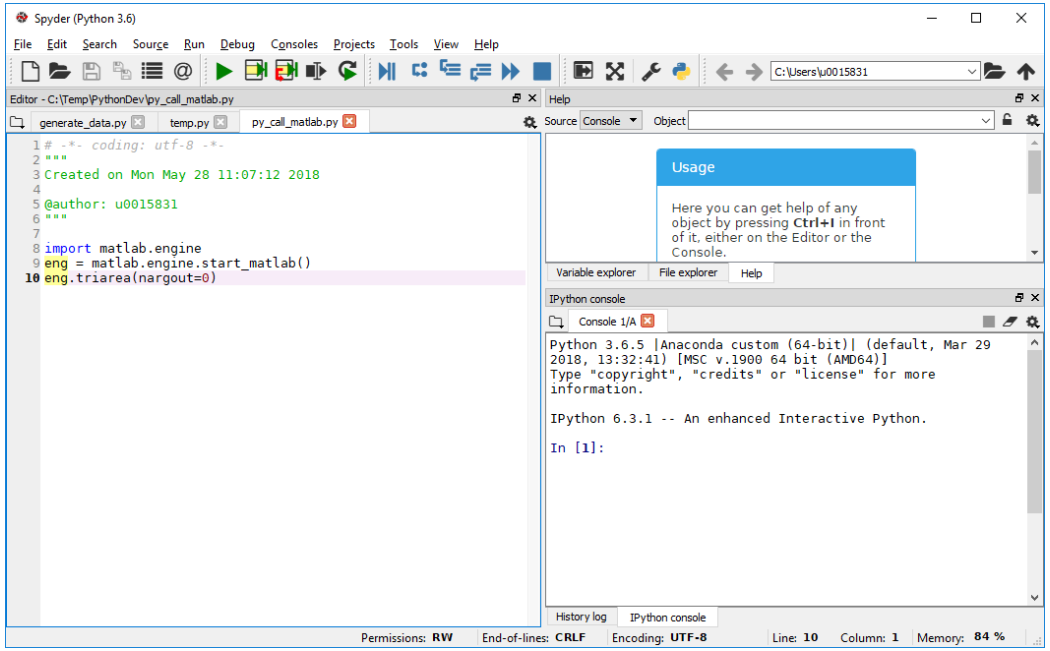
- Integrate different aspects of programming and running code.
- SPyDER: "*Scientific Python Development EnviRonment*“ https://www.spyder-ide.org/
- Several tools in one integrated environment (cfr MATLAB desktop)
  - a code editor
  - IPython interpreter / console
  - variable inspector
  - control icons
- Documentation: https://docs.spyder-ide.org/current/index.html



IDE: Spyder

- Spyder for code development.
  - Command window: `spyder`

- Magic commands apply
  - Clear Console:
    - `%cls`
  - Clear all variables from Variable Explorer (reset the namespace):
    - `%reset`
  - With `automagic on`, % prefix not needed



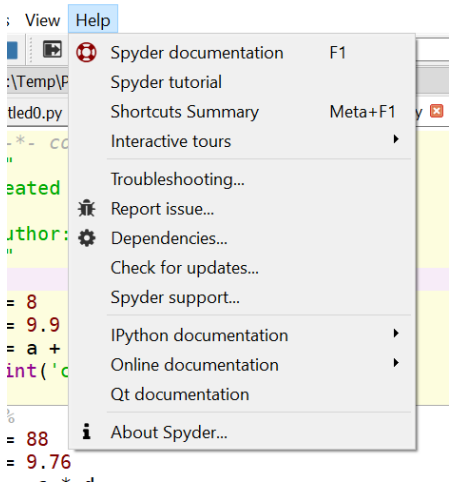
Spyder Help

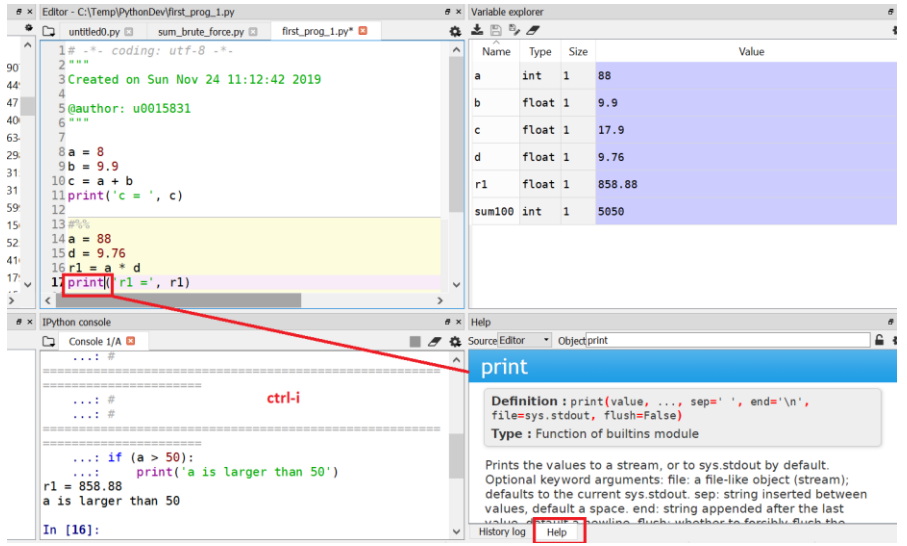
- Help on Spyder from Help menu
- Help related to Python
  - Select a command and press `ctrl-I`
    - Information opens in help window
  - Enter object in help window
- help(command) in console

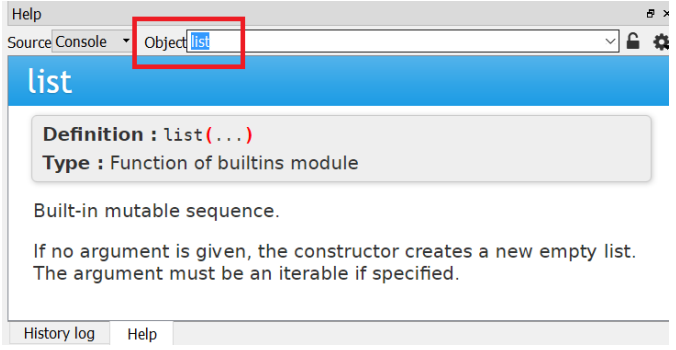
```
In [18]: help(print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n',
file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by
default.
    Optional keyword arguments:
```

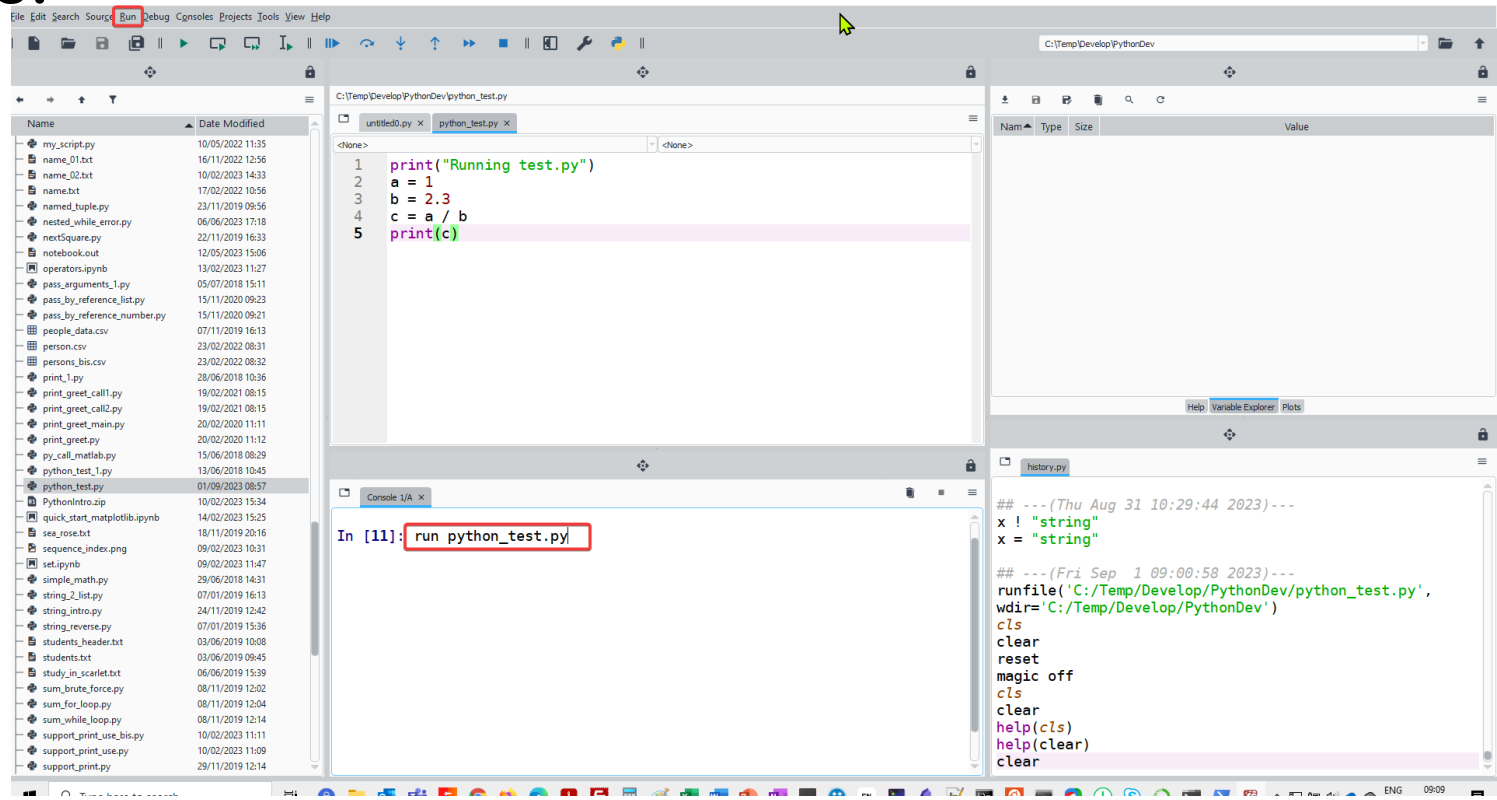






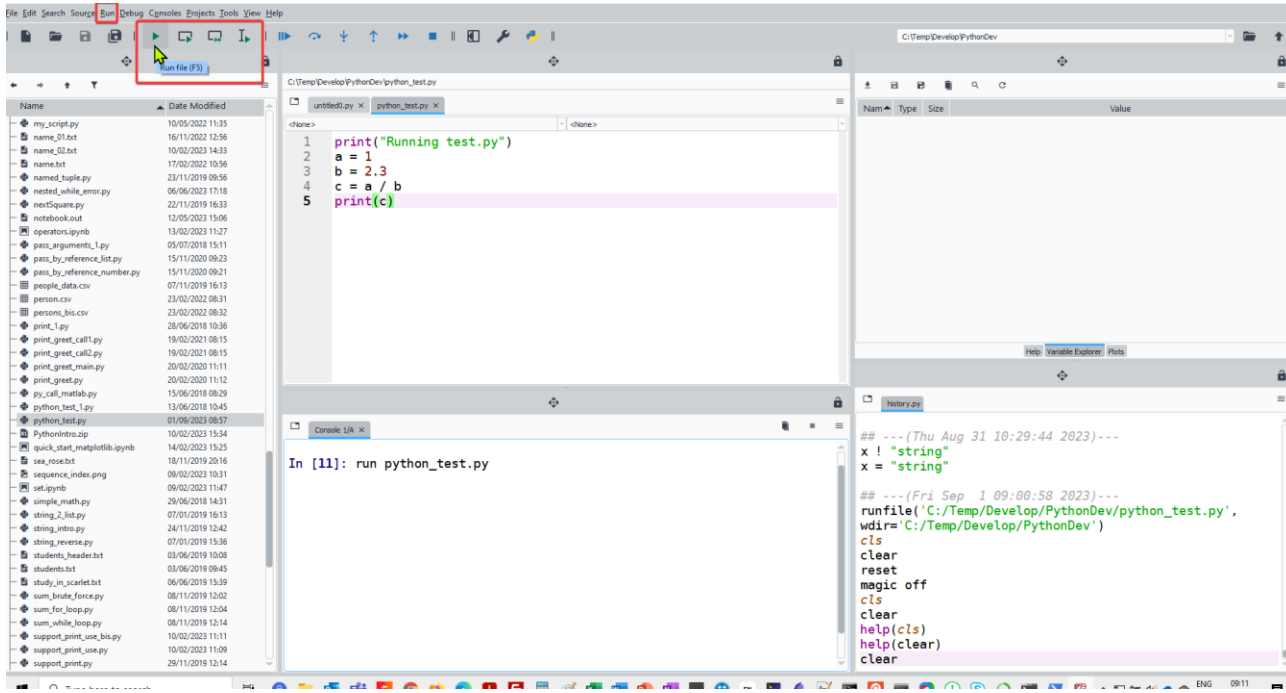
Running scripts in Spyder console

- Run a .py file from the console
  - `run script.py`
- Tab autocompletion works!



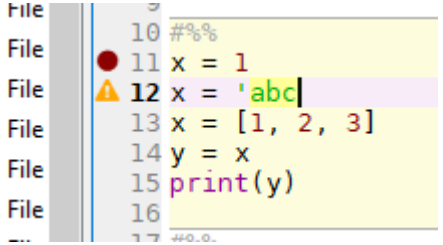
Running scripts in Spyder

- Run scripts either using the icons or through the Run menu.
- *Run selection or current line* will run a highlighted portion of the script.
- Create **cells** by enclosing chunks of code with lines consisting of #%%
  Run cell/green arrow with a box runs the cell.
- *File: first_prog_1.py*



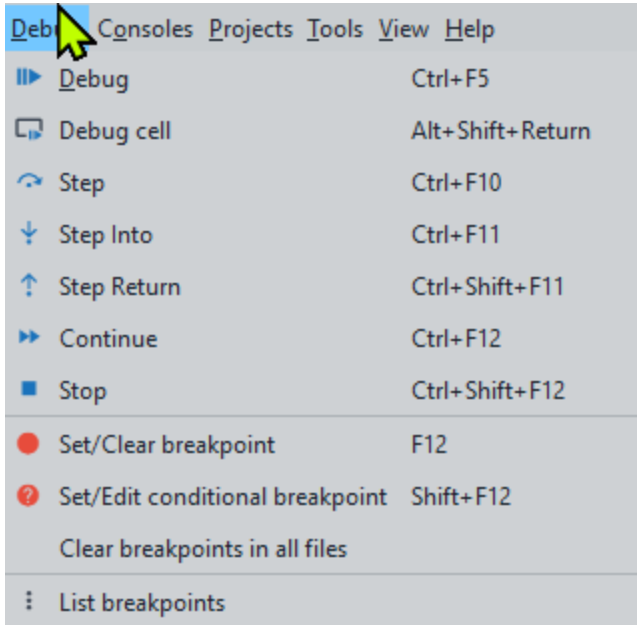
Running scripts in Spyder

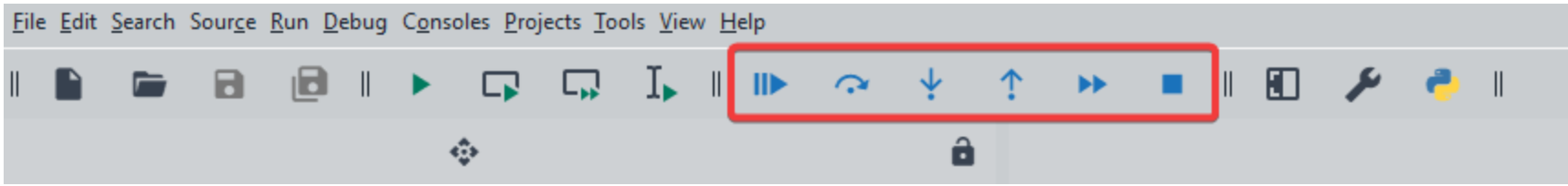
- A yellow triangle beside a line indicates a syntax error or potential problem.
- Tab completion for names familiar to it. It can show a list of members of a package for your selection, and when you have chosen a function it can show you a list of its arguments.



Debugging in Spyder

- Debugging tool
  - The Debug menu at the top contains a list of all the options for debugging
  - The navigation bar also has the icons associated with those tasks.



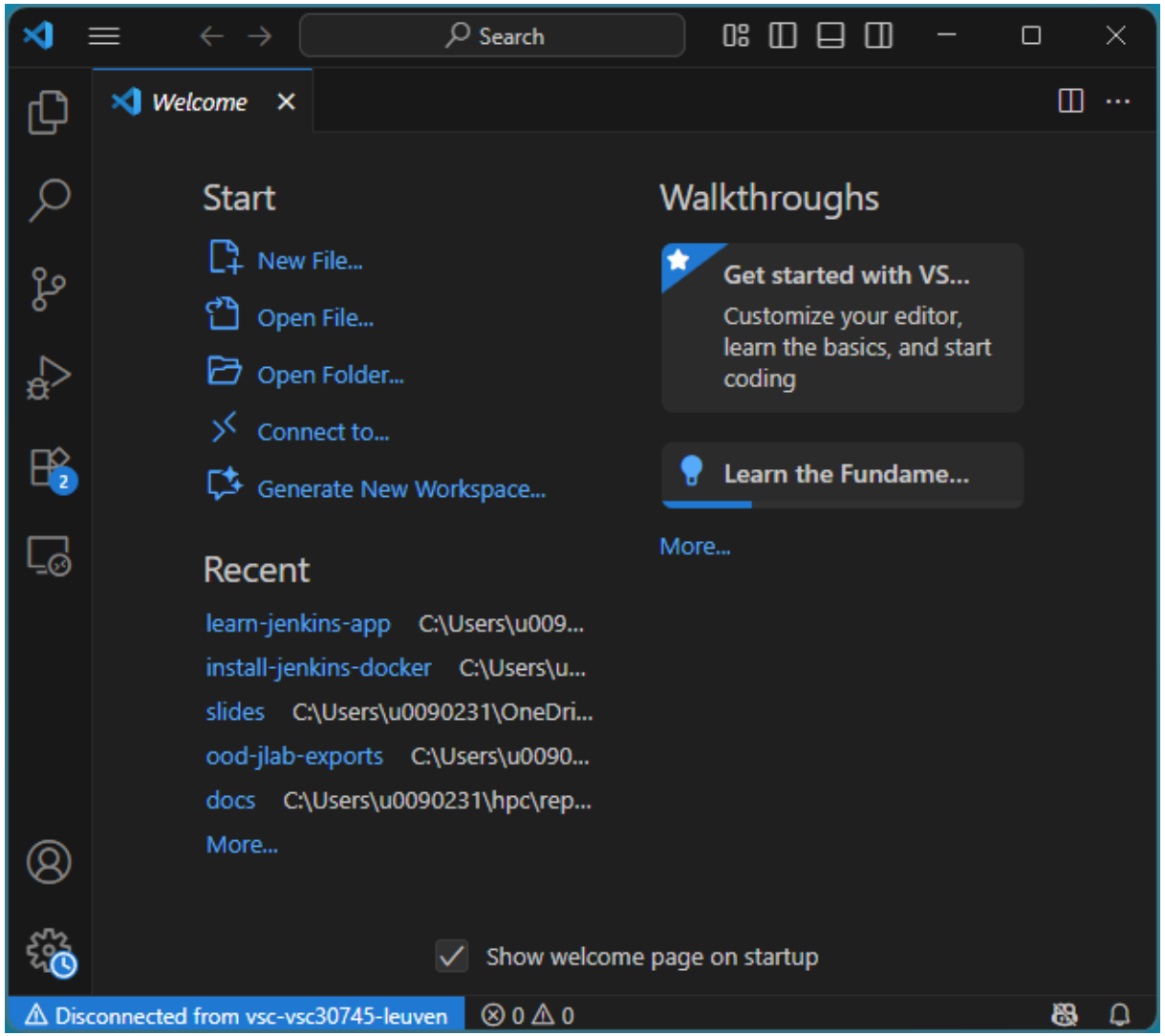


Visual Studio Code

Another IDE

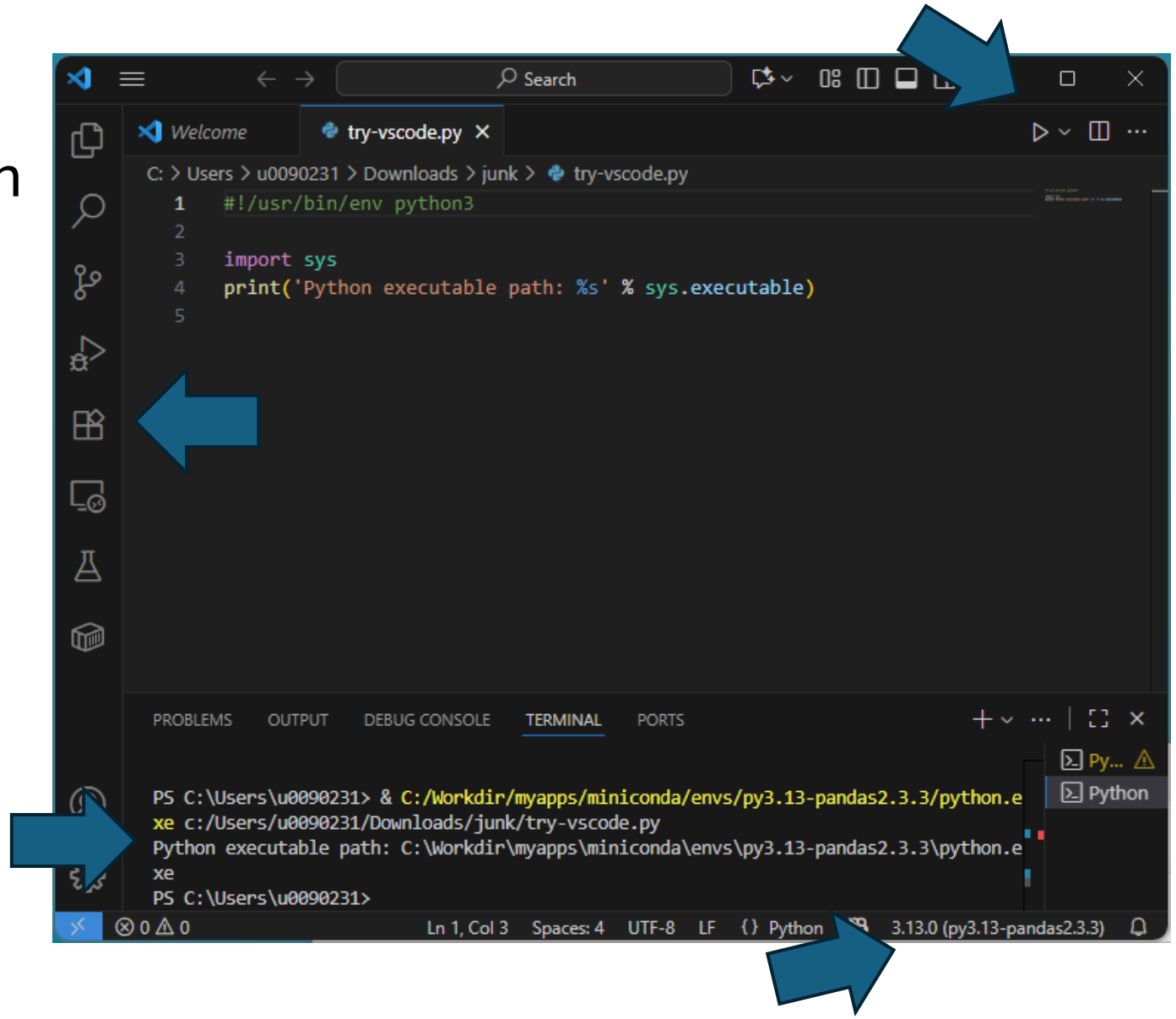
IDE: Visual Studio Code (VSCode)

- VSCode integrates different aspects of programming and running code.
- Microsoft & 3rd party: Support for extensions (e.g. Python, C/C++, Java, Julia ..., Git, Docker, ...)
- Github Copilot integration



Setup

- Install Python extension from Microsoft (and restart VSCode)
- Choose Python interpreter (bottom right tray) and Pick your Conda env
- Click on the Play button (top right), or
  choose lines -> Shift+Enter

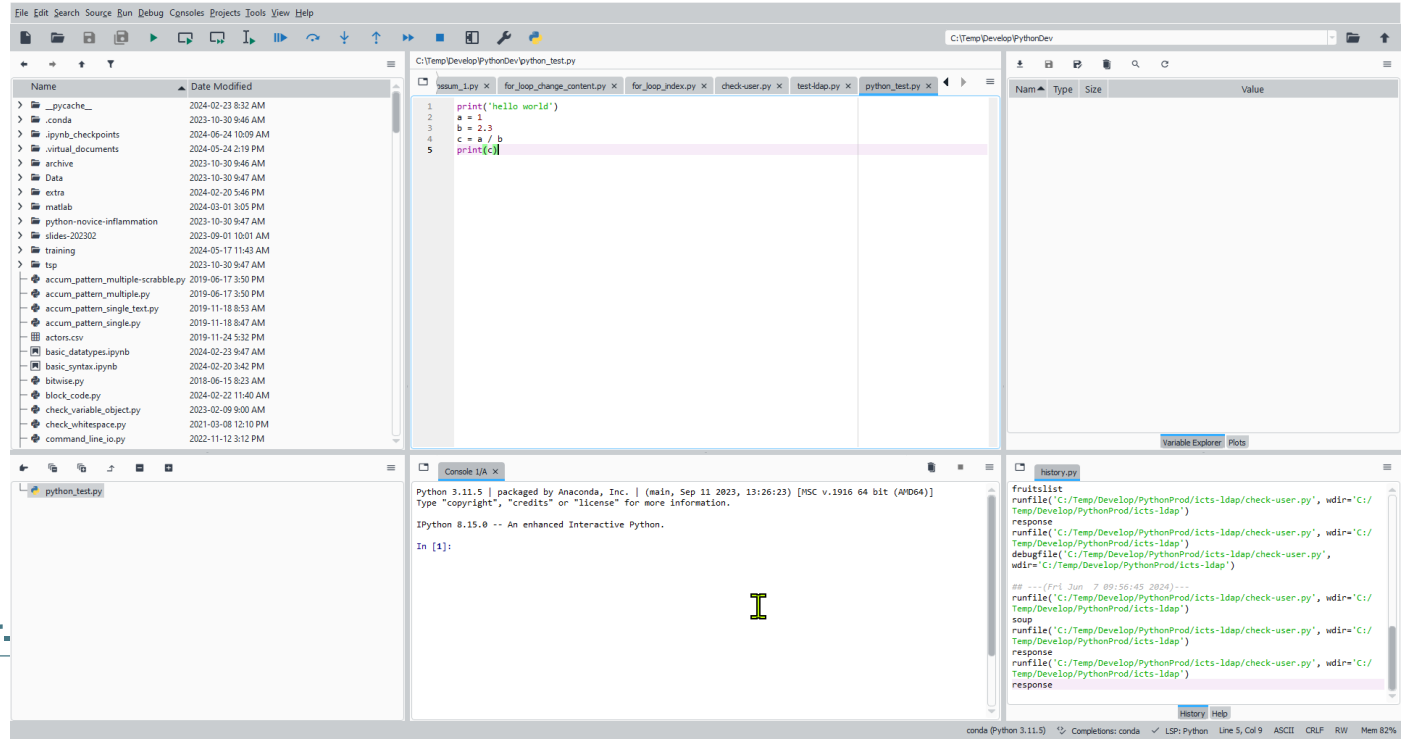


Spyder

Another IDE

IDE: Spyder

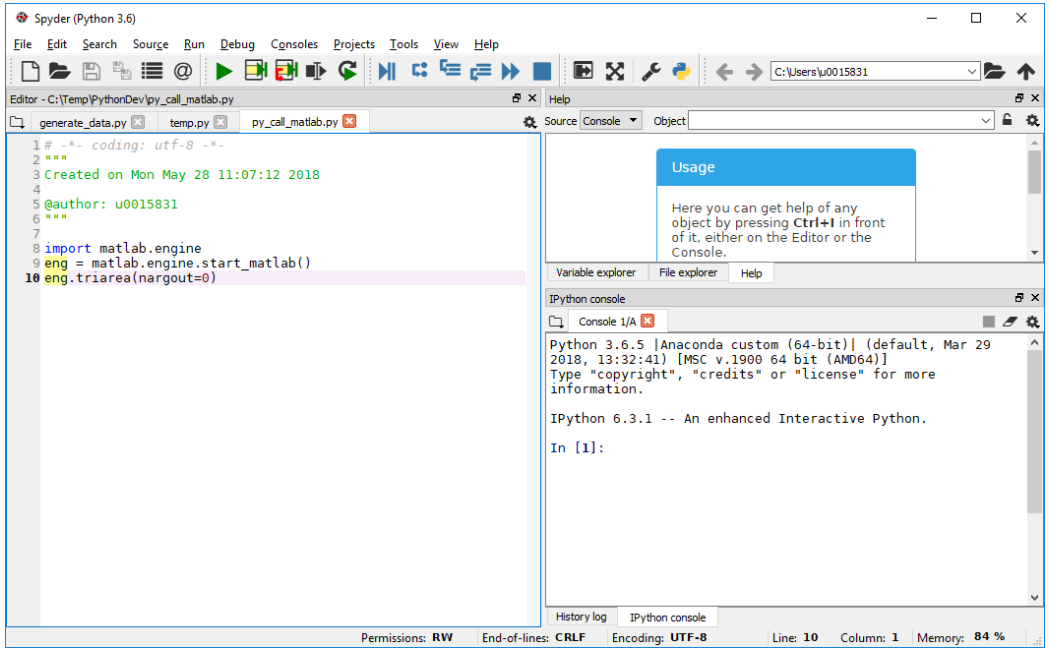
- Integrate different aspects of programming and running code.
- SPyDER: "*Scientific Python Development EnviRonment*“ https://www.spyder-ide.org/
- Several tools in one integrated environment (cfr MATLAB desktop)
  - a code editor
  - IPython interpreter / console
  - variable inspector
  - control icons
- Documentation: https://docs.spyder-ide.org/current/index.html



IDE: Spyder

- Spyder for code development.
  - Command window: `spyder`

- Magic commands apply
  - Clear Console:
    - `%cls`
  - Clear all variables from Variable Explorer (reset the namespace):
    - `%reset`
  - With `automagic on`, % prefix not needed



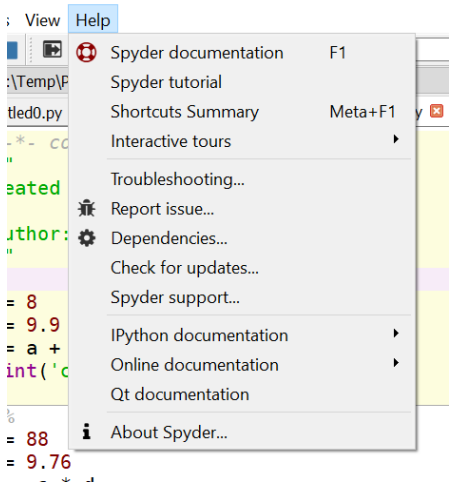
Spyder Help

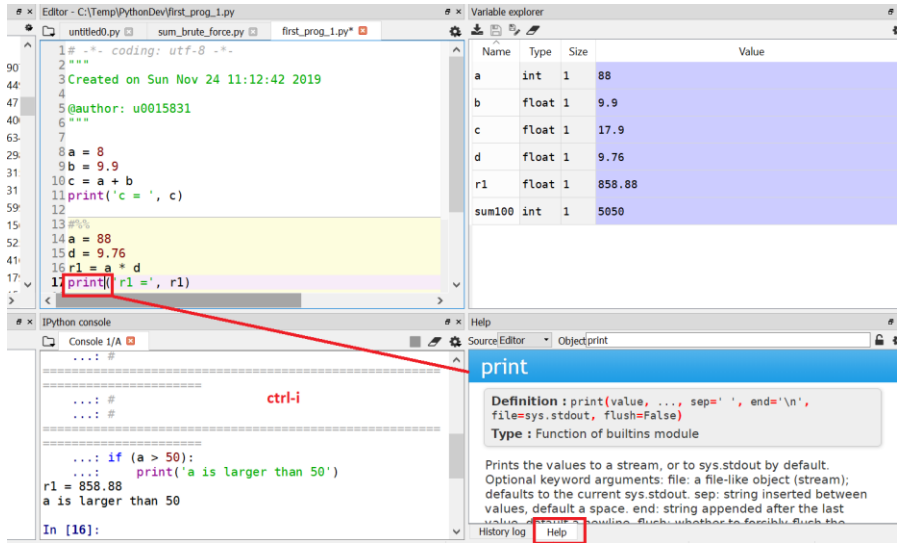
- Help on Spyder from Help menu
- Help related to Python
  - Select a command and press `ctrl-I`
    - Information opens in help window
  - Enter object in help window
- help(command) in console

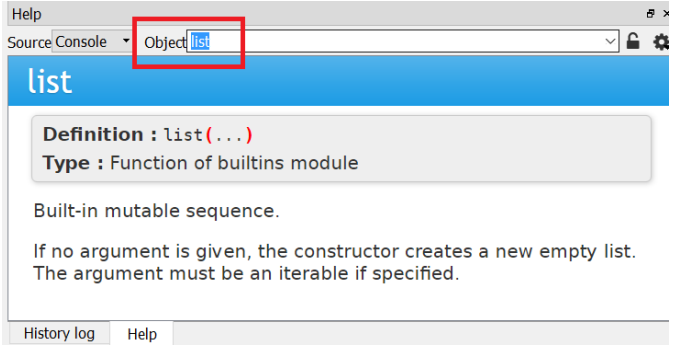
```
In [18]: help(print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n',
file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by
default.
    Optional keyword arguments:
```

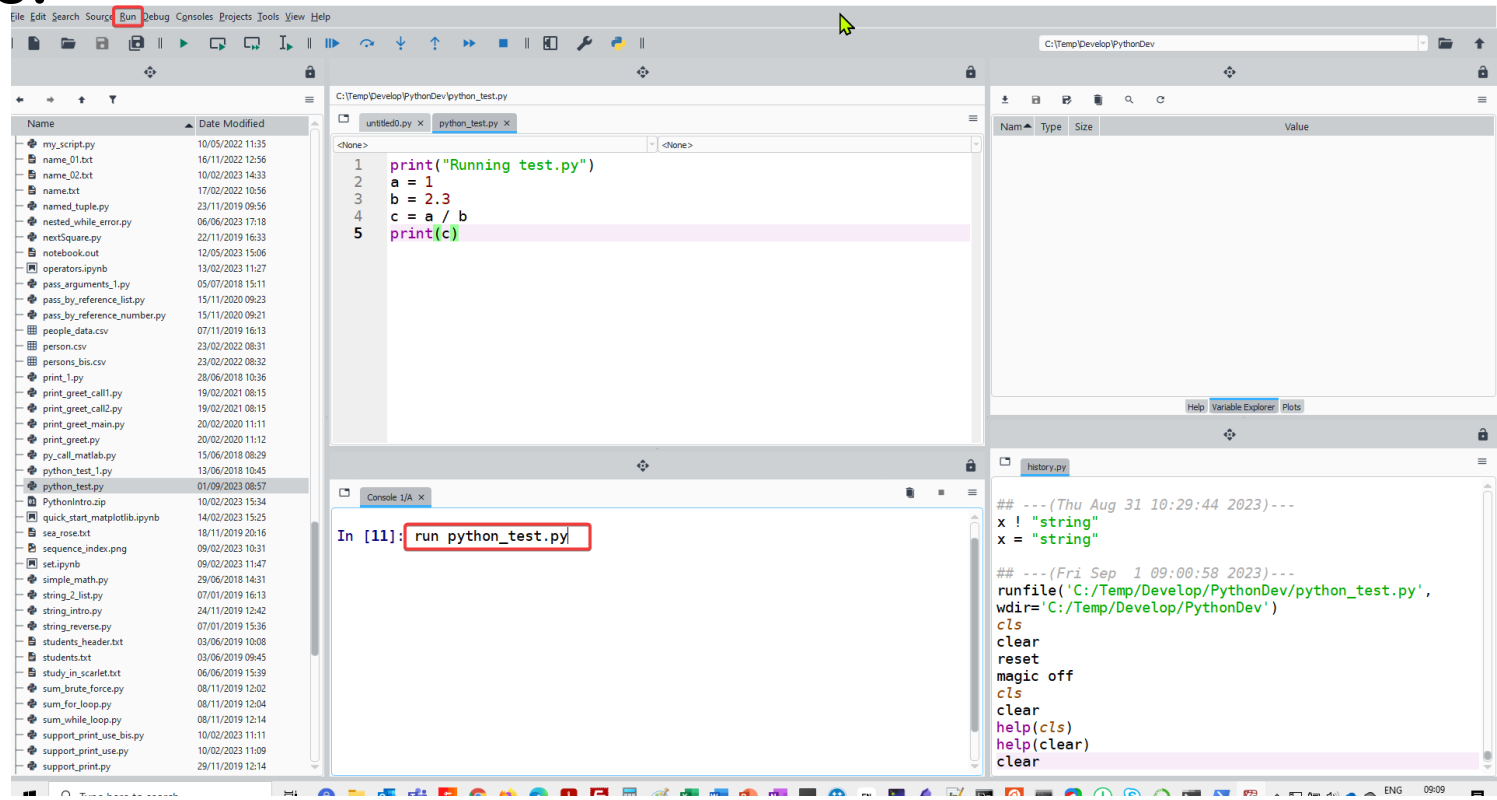






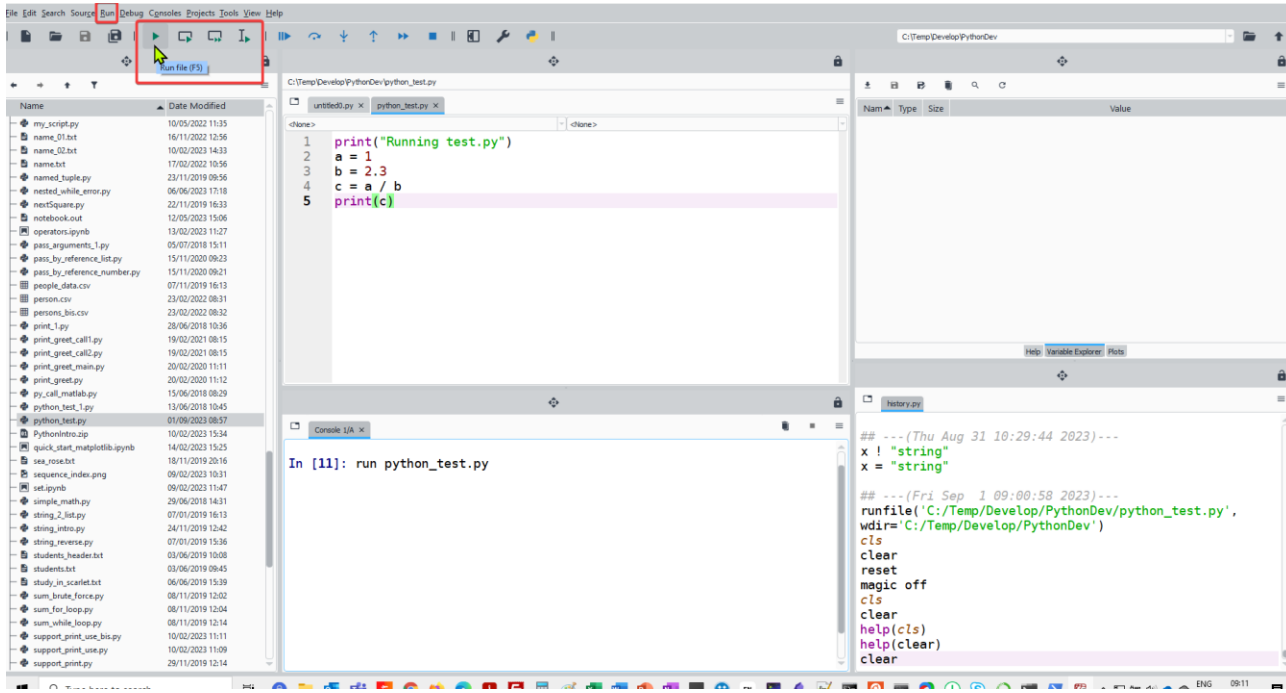
Running scripts in Spyder console

- Run a .py file from the console
  - `run script.py`
- Tab autocompletion works!



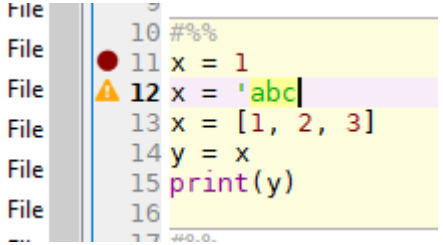
Running scripts in Spyder

- Run scripts either using the icons or through the Run menu.
- *Run selection or current line* will run a highlighted portion of the script.
- Create **cells** by enclosing chunks of code with lines consisting of #%%
  Run cell/green arrow with a box runs the cell.
- *File: first_prog_1.py*



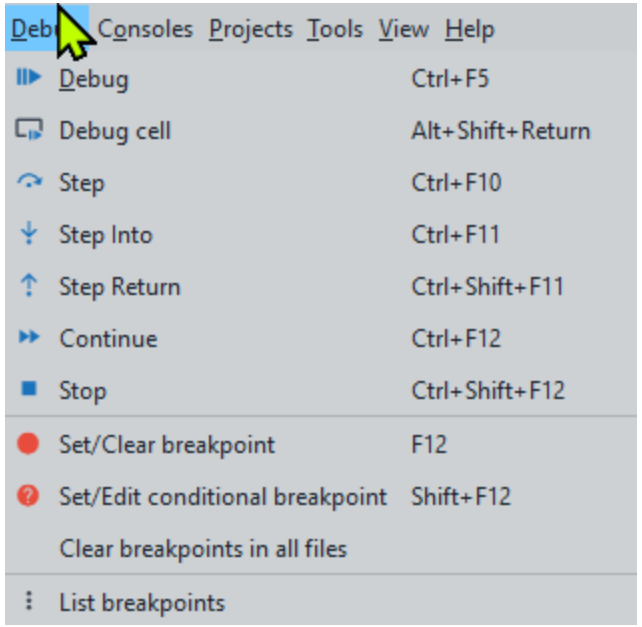
Running scripts in Spyder

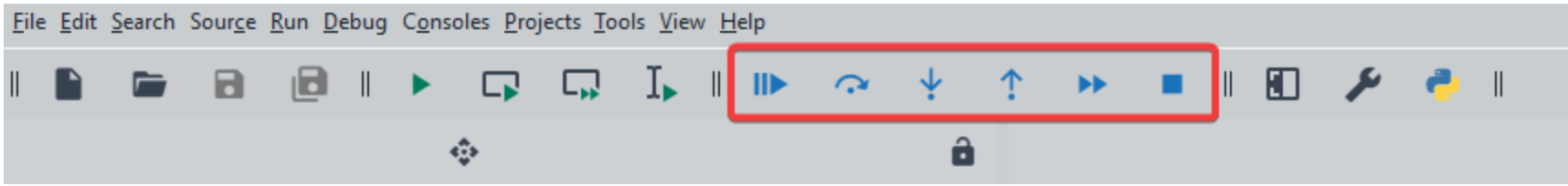
- A yellow triangle beside a line indicates a syntax error or potential problem.
- Tab completion for names familiar to it. It can show a list of members of a package for your selection, and when you have chosen a function it can show you a list of its arguments.



Debugging in Spyder

- Debugging tool
  - The Debug menu at the top contains a list of all the options for debugging
  - The navigation bar also has the icons associated with those tasks.





Jupyterlab / Jupyter notebook

getting_started_jupyter.ipynb

(Jupyter) notebook

- A nice idea popularized by Mathematica is a “notebook” interface, where you can run and re-run commands
- Easily mix code with comments, and mix code with the results of that code; including graphics, …
- https://realpython.com/jupyter-notebook-introduction/
- https://docs.anaconda.com/ae-notebooks/4.2.2/user-guide/basic-tasks/apps/jupyter/
- https://towardsdatascience.com/5-reasons-why-jupyter-notebooks-suck-4dc201e27086

(Jupyter) notebook

- Excellent for
  - Explorative programming
  - Data exploration
  - Communication, especially across domains
  - Combine code, graphs, results and text for education
- Problems?
  - What was (re-)executed, what not?
  - Version control?

- https://github.com/gjbex/training-material/blob/master/Python/python_intro.pptx

Jupyter notebook vs Jupyterlab

- JupyterLab and Jupyter Notebook serve as interactive computing environments, they differ in their user interface, functionality, and flexibility.
- Jupyter Notebook has a simpler, more lightweight interface.
- JupyterLab offers a more versatile and feature-rich interface , it gives a more IDE-like experience.

JupyterLab

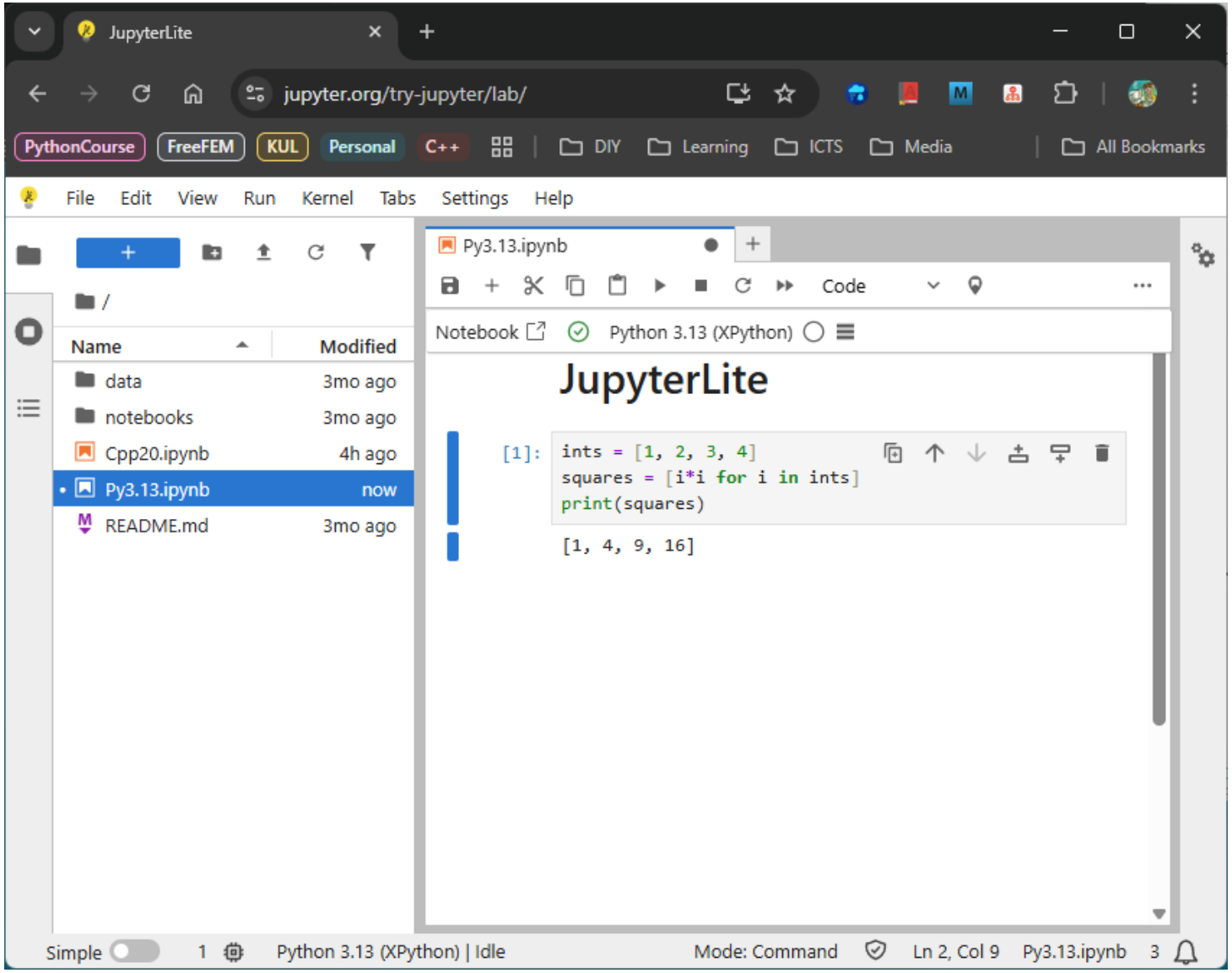
- Interactive Development Environment for working with notebooks, code and data.
- Next-generation user interface for Project Jupyter.
- It offers all the familiar building blocks of the classic Jupyter Notebook (notebook, terminal, text editor, file browser, rich outputs, etc.) in a flexible and powerful user inteface.
  Eventually, JupyterLab will replace the classic Jupyter Notebook.
  - File support: JupyterLab provides built-in support for a wider range of file formats and includes integrated terminals and code consoles.
  - Extensibility: JupyterLab is designed to be extensible, enabling users to install additional extensions and customize the environment to meet their specific needs.
  - Compatibility: JupyterLab is compatible with existing Jupyter Notebook files and kernels, allowing users to transition smoothly between the two interfaces.
- https://realpython.com/using-jupyterlab/
- https://saturncloud.io/glossary/jupyter-notebook-vs-jupyterlab/
- https://www.youtube.com/watch?v=yjjE-MJD5TI (Cornell CAC JupyterLab tutorial)

JupyterLab: how to launch?

- VSCode natively supports Jupyter Notebook files `*.ipynb`
- Install JupyterLab in conda:
  `conda install jupyterlab -c conda-forge`
- Anaconda Navigator:
  - Start menu
  - Launch **JupyterLab**
- Anaconda prompt
  - Open terminal and navigate to the **directory** where you would like to save your notebook
  - Note: JupyterLab treats the directory from which it is launched as the top-level directory
  - `jupyter lab`
  - Options possible `jupyter lab --browser=firefox`

Python Environment for This Training

- Jupyterlite: https://jupyter.org/try-jupyter/lab/
- JupyterLab in the browser
- Choose the ‘Python 3.13 (XPython)’ kernel
- Download your `*.ipynb` notebooks before closing

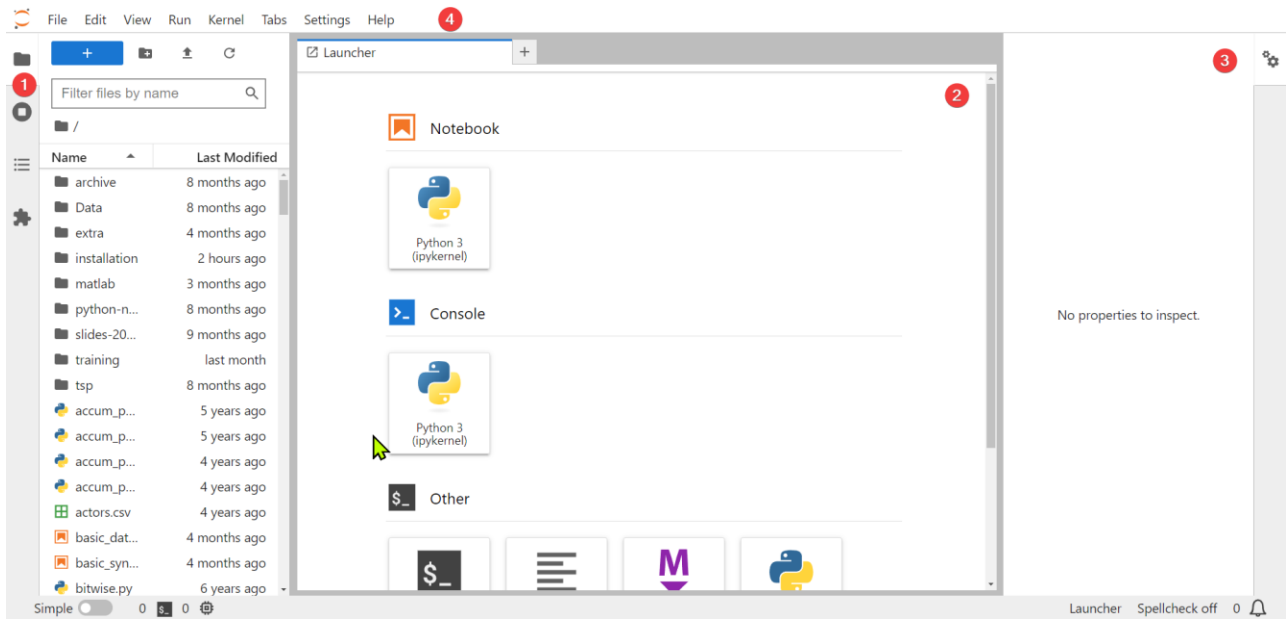


Jupyterlab interface

https://jupyterlab.readthedocs.io/en/stable/user/interface.html

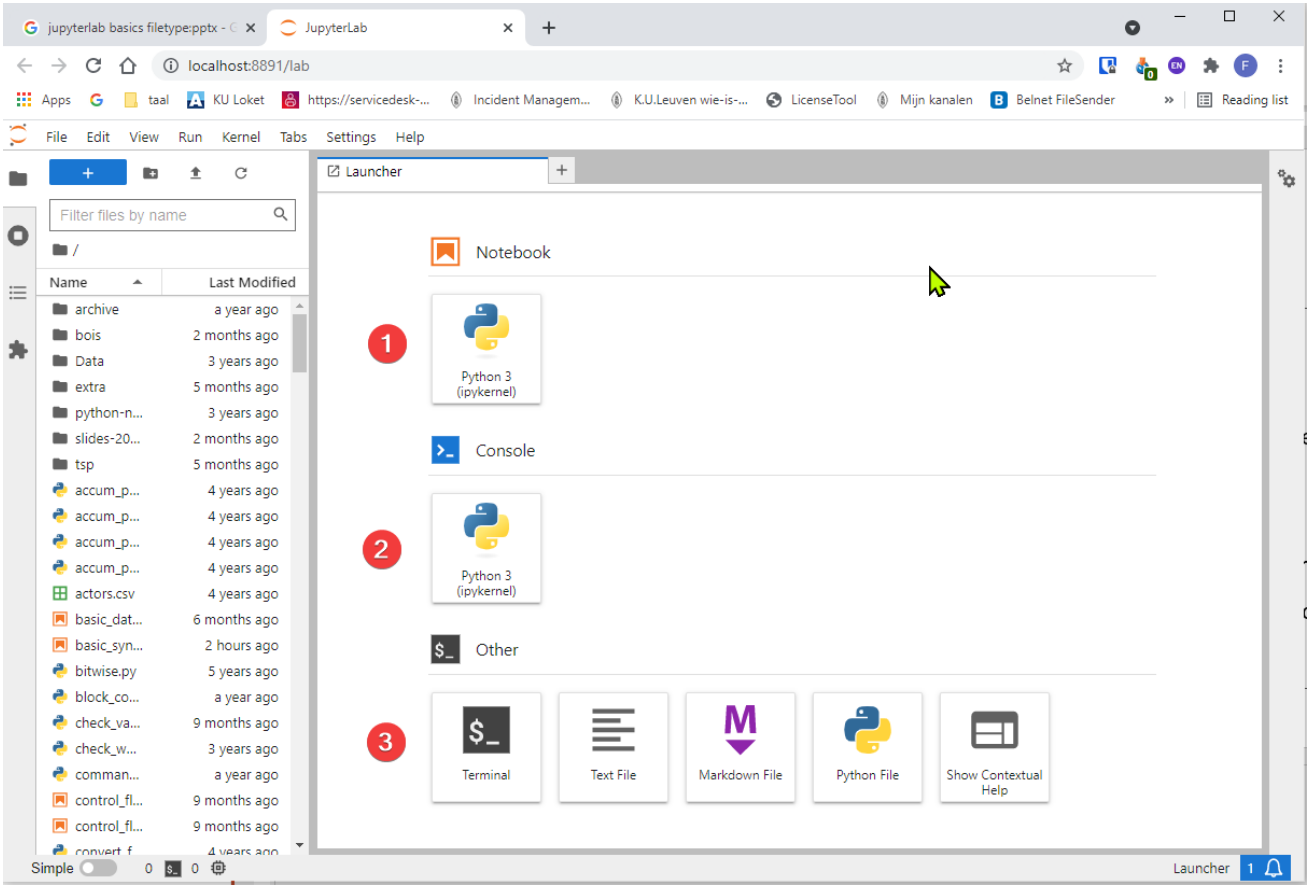
Interface can be tweaked

1. Left panel:
    1. File browser
        1. New launcher
        2. New folder
        3. Upload files
    2. Running terminals and kernels
2. Center panel: main work area containing tabs of documents and activities
3. Right panel: shows various properties



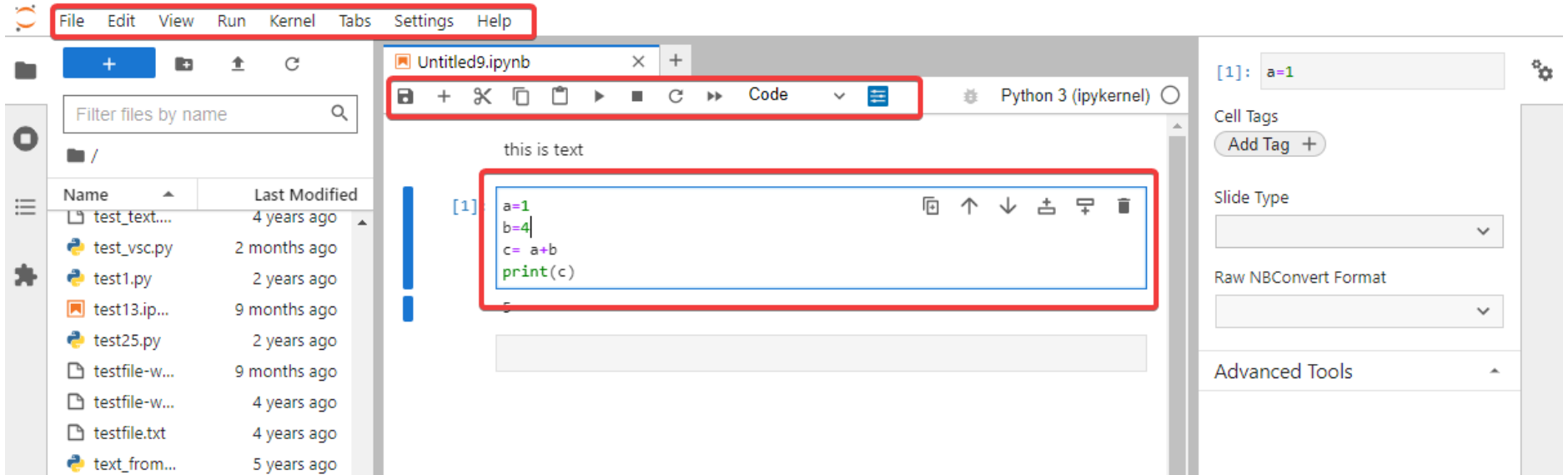
JupyterLab interface

1. Launch your notebook
2. Launch Python kernel
3. Launch another application (i.e. terminal)



JupyterLab interface

- Menu bar: different options that may be used to manipulate the way the notebook functions.
- Toolbar: a quick way of performing the most-used operations within the notebook.
- Cell: the notebook cell.



JupyterLab Notebook

- The notebook consists of a sequence of cells.
  - A cell is a multiline text input field
  - The execution behaviour of a cell is determined by the cell's type.
- 3 types of cells:
  - **Code** cells allow you to edit and write new code, with full syntax highlighting and tab completion. The programming language depends on the kernel chosen.
  - **Markdown** cells allow to alternate descriptive text with code
  - **Raw** cells provide a place in which you can write output directly. Raw cells are not evaluated by the notebook.

JupyterLab shortcuts

The essential shortcuts:

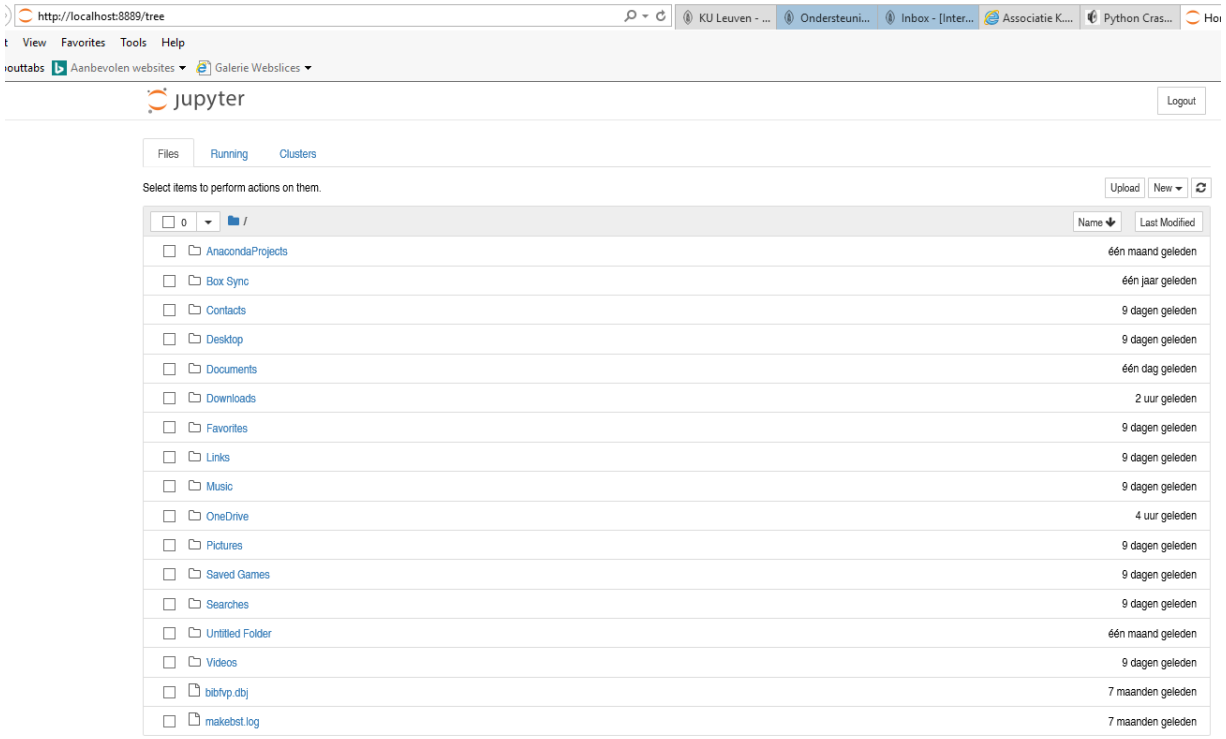
- **Shift-Enter**: run cell and move to the next
  - Execute the current cell, show any output, and jump to the next cell below. If Shift-Enter is invoked on the last cell, it creates a new cell below. This is equivalent to clicking the Cell, Run menu item, or the Play button in the toolbar.
- **Ctrl-Enter**: run cell and stay in that cell
  - Execute the current cell, show any output.
- **Esc**: Command mode.
  - In command mode, you can navigate around the notebook using keyboard shortcuts.
- **Enter**: Edit mode.
  - In edit mode, you can edit text in cells

Jupyterlab shutdown

- Make sure everything is saved
- Download your notebook(s) if needed
- Use File → Shut Down to close the application
- Close the browser (tab)

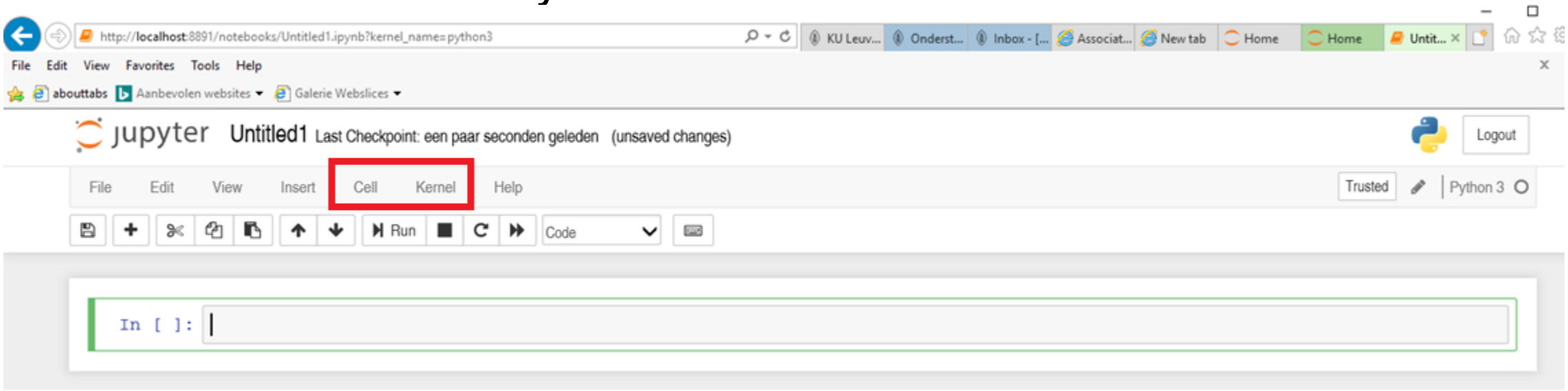
Jupyter notebook

- Notebook Dashboard, specifically designed for managing your Jupyter Notebooks.
- Use it as the launchpad for exploring, editing and creating your notebooks.



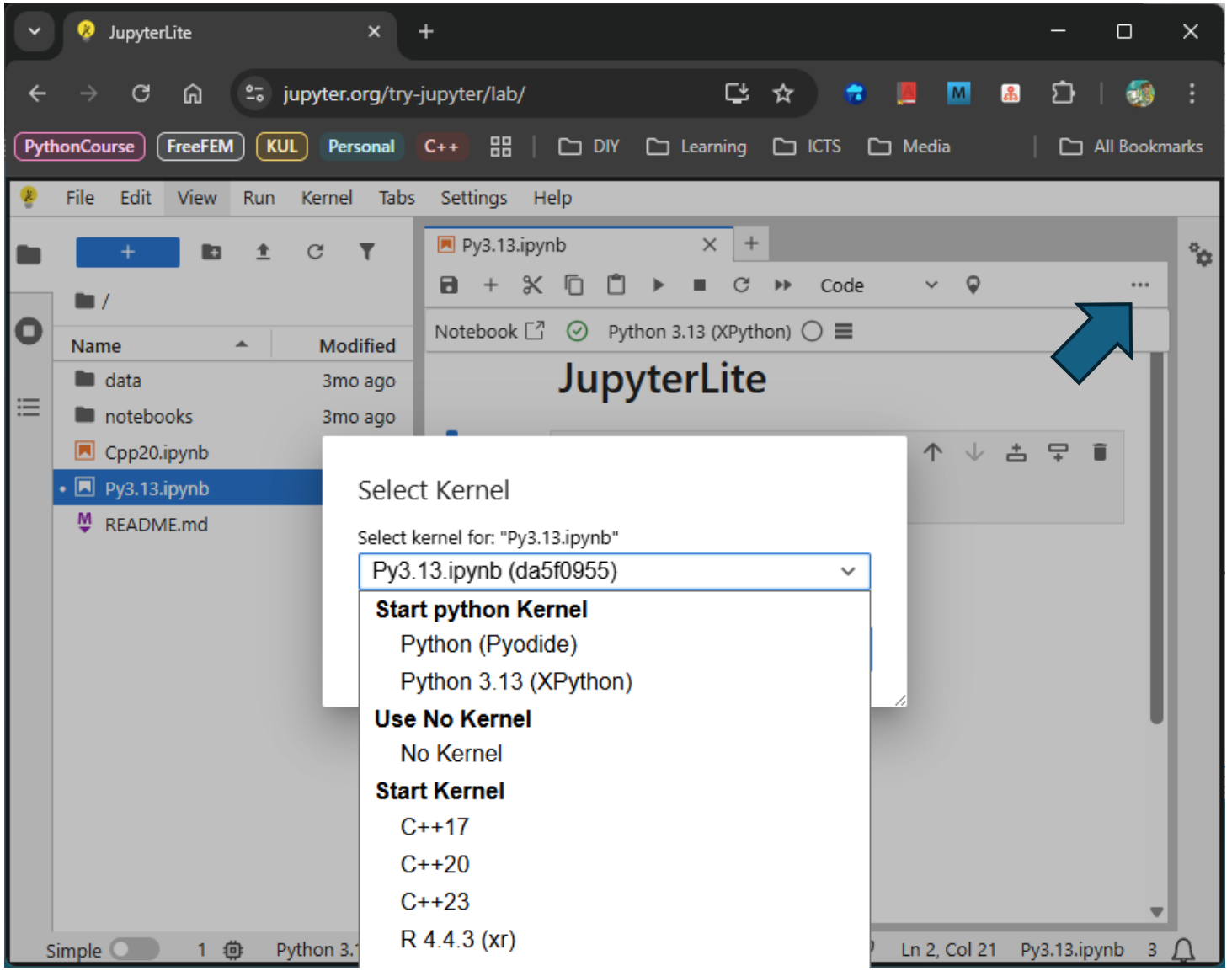
Jupyter notebook

- Jupyter is essentially an advanced word processor.
- A kernel is a "computational engine" that executes the code contained in a notebook document.
- A cell is a container for text to be displayed in the notebook or code to be executed by the notebook's kernel.



Jupyter notebook

- Browse to the folder in which you would like to create your first notebook,
- Click the "New" drop-down button in the top-right and
- Select "Python 3.13" (or the version of your choice).

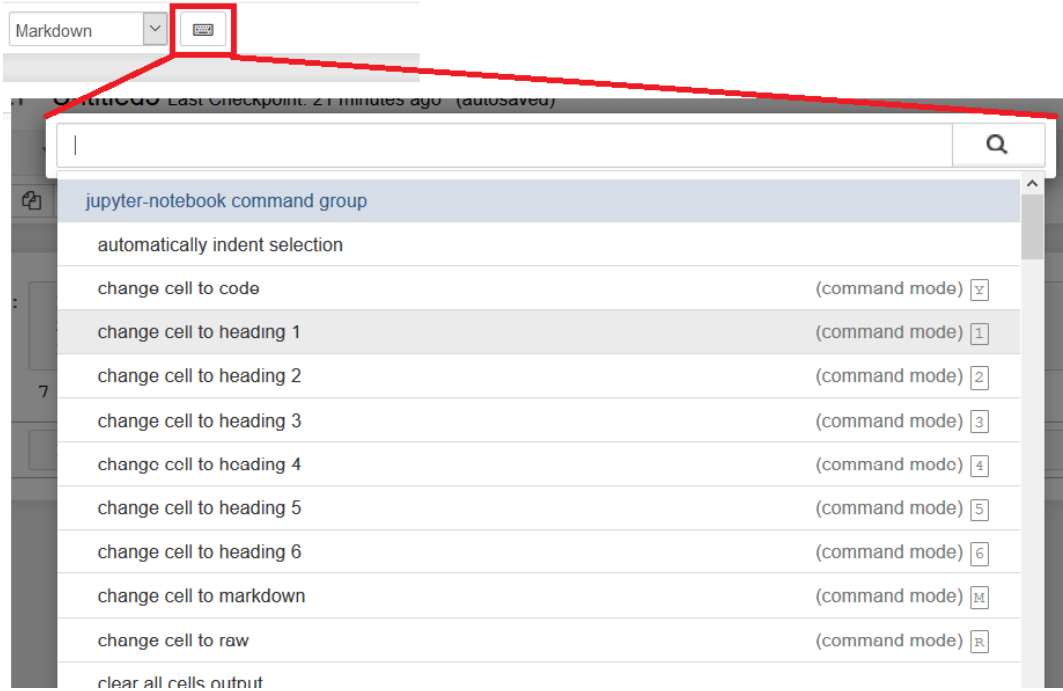


Jupyter: basics of editing

- Jupyter notebook: sequence of cells
  - **Code**
    - Label “In [ ]” in front of the code
    - a * will appear when executing
    - replaced by a number that always increases by one with each cell execution. This allows for keeping track of the order in which the cells in the notebook have been executed.
  - **Markdown**
- Important shortcut: ctrl+Enter (execute cell)
- Color code
  - Blue bar on the left: active cell in command mode
  - Click in cell, changes in edit mode – Green bar
- Jupyter will periodically autosave the notebook

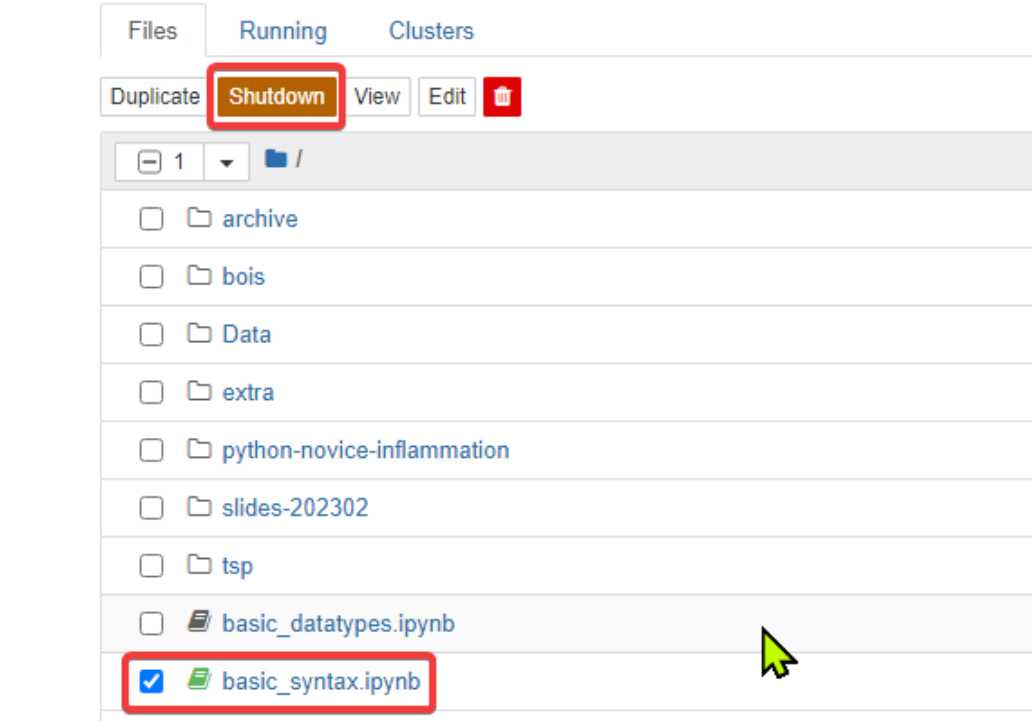
Jupyter: basics of editing

- try to know the basic shortcuts
- **Command mode** shortcuts:
  - Basic navigation: enter, **shift-enter**, up/k, down/j
  - Saving the notebook: s
  - Change Cell types: y, m, 1-6, t
    - m to change the current cell to Markdown,
    - y to change it back to code
  - Cell creation: a, b
    - a to insert a new cell above the current cell,
    - b to insert a new cell below
  - Cell editing: x, c, v, d, z
    - c copy selected cells
    - x cut selected cells
    - v paste copied cells
    - d + d (press the key twice) to delete the current cell
    - z undo cell deletion



Close and Shutdown Jupyter Notebook

- Close Jupyter Notebook files
  - Close the browser tab displaying the notebook, but you still need Shutdown the notebook from the dashboard.
  - To Shutdown a Jupyter Notebook file (.ipynb), click in the checkbox to left of the filename. An orange button (**Shutdown**) appears in the dashboard menu; click on it to Shutdown any file that is checked in the list.
- Shutdown the Jupyter Notebook Local Server
  - After all of your notebooks are closed and shut down, you can end your Jupyter Notebook session by clicking on the **Quit** button at the top right of the dashboard.
  - Close the terminal by typing the command exit



Jupyter: some tips

- Run a notebook on the command line with `ipython`
- Jupyter notebook tips https://www.dataquest.io/blog/jupyter-notebook-tips-tricks-shortcuts/
- https://www.dataquest.io/blog/jupyter-notebook-tutorial/
- https://jupyter4edu.github.io/jupyter-edu-book/
- https://reproducible-science-curriculum.github.io/workshop-RR-Jupyter/
- Change the default startup directory
  - https://stackoverflow.com/questions/35254852/how-to-change-the-jupyter-start-up-folder
- Change the default browser
  - https://support.anaconda.com/customer/en/portal/articles/2925919-change-default-browser-in-jupyter-notebook

Scripts vs Notebooks

Scripts vs notebooks: script

- Python script:
  - Plain text file ending with the .py extension containing the program
  - Created in editor, IDE
  - Executed from the command line
- Jupyter Notebook:
  - Stored in notebook files, having the .ipynb extension.
  - Multiple cells. Each cell can contain either a block of Python code or plain text.

Scripts vs notebooks: script

- Pros:
  - Scripts are reliable and the most common way to write Python code.
  - Top-down execution makes it less confusing to debug and reason through the code.
  - Scripts support modularity. Variables and functions inside a Python script can be imported.
  - Can be placed in version control
  - Minimal setup is required (you only need a text editor).
  - There are many text editors and IDEs with tons of features to choose from.
- Cons:
  - Scripts are plain text files. Formatted text or figures cannot be added to them.
  - No output is saved anywhere. The script must be executed to see messages, outputs, and results.

Scripts vs notebooks: notebook

- Pros:
  - Code blocks can be surrounded by helpful notes, figures, and links.
  - Notebooks provide nonlinear execution. Code cells can be run independently from one another.
  - Output (messages, plots, etc.) appear automatically under each cell
  - Exploratory computing, prototyping
  - Sharing results
- Cons:
  - Nonlinear execution can make debugging confusing, especially if you lose track of which cells were executed or not.
  - Version control can be a problem
  - Require installing the jupyter-notebook package
  - Notebooks must be served and accessed through a web browser, making them slightly harder to use than scripts.

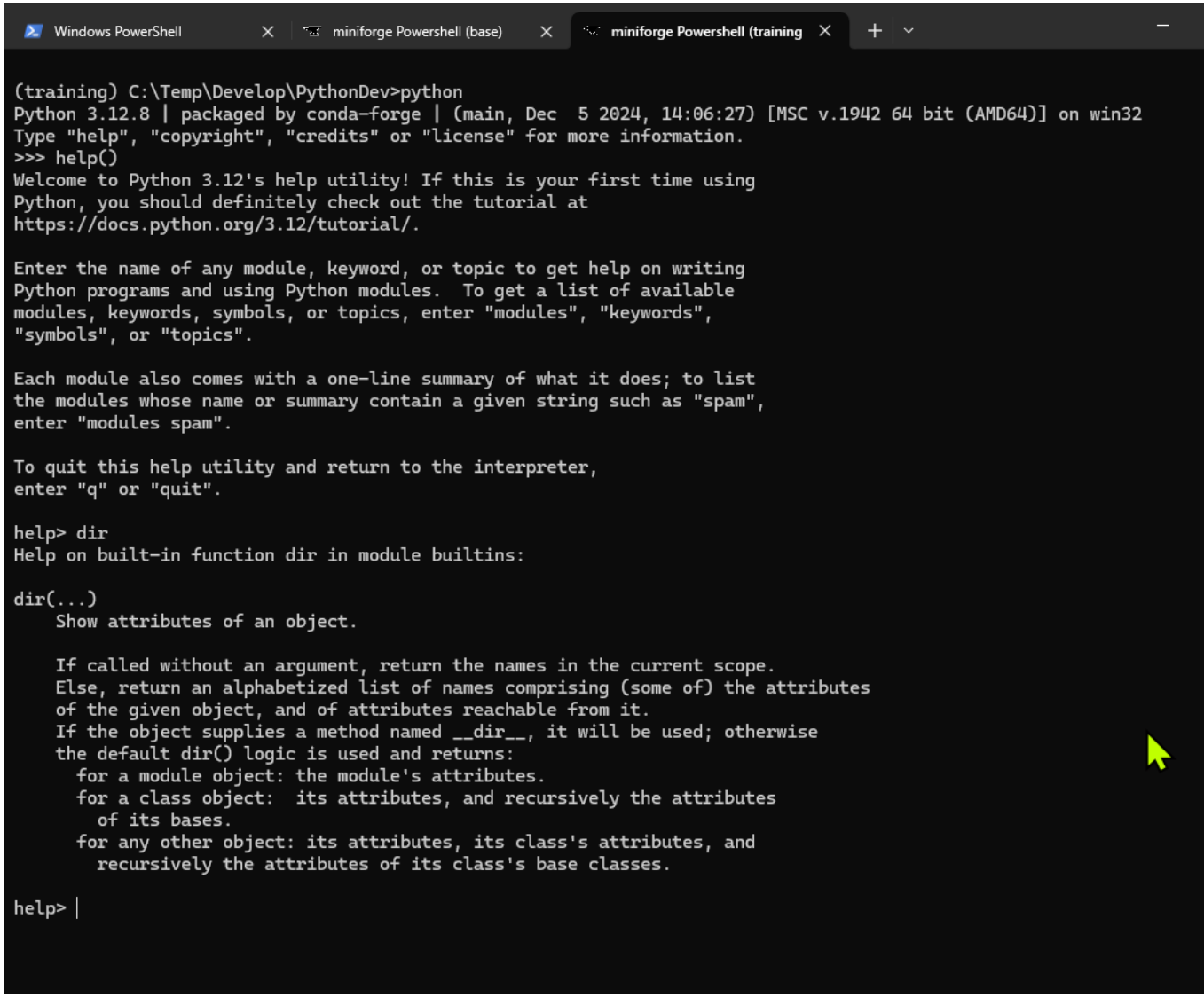
What to use?

- Compromise between Python scripts and Jupyter Notebooks
- Start out with a Notebook: explore ideas and get a clearer picture of what is needed.
- As the ideas grow clearer:
  - Put the code in a Python script
  - Put effort in using functions, modules
- *Example?: prompt ai for code*

Getting Help

Getting Help

- Python comes with a built-in help system. This means that you don't have to seek help outside of Python itself.
- `help()`: running the function without an argument, the interactive Python's help utility will be started
  - `q` to quit
  - Type the command to get the help information



Getting Help

- Passing an Object to help()
  - `help(print)`
  - `help(str.upper)`
- Passing a string to help()
  - pass a string as an argument, the string will be treated as the name of a function, module, keyword, method, class, or a documentation topic and the corresponding help page will be printed.
- When needing help about a function from a certain Python library:
  - First import the library.
  - Ask to get the documentation for the function defined in the Python library.

```
>>> help(print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file:  a file-like object (stream); defaults to the current sys.stdout.
    sep:   string inserted between values, default a space.
    end:   string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.

>>> help(str.upper)
Help on method_descriptor:

upper(self, /)
    Return a copy of the string converted to uppercase.

>>>
```

```
>>> help(log)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'log' is not defined
>>> import math
>>> help(log)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'log' is not defined
>>> help(math.log)
Help on built-in function log in module math:

log(...)
    log(x, [base=math.e])
    Return the logarithm of x to the given base.

    If the base not specified, returns the natural logarithm (base e) of x.
```

Getting Help

- Python website provides in depth online documentation: https://docs.python.org/3/index.html
- Python website provides a comprehensive tutorial that has many examples: https://docs.python.org/3/tutorial/index.html